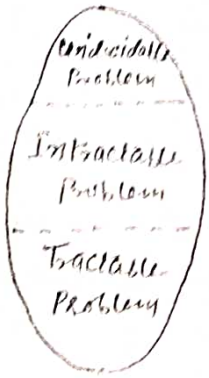
Unit-V

NP-Completeness

Different categories of problems are



Classification of Problems:

(i) P:

P stands for Polynomial.

Problems that can be solved in Polynomial Time. More specifically, they are the problems that can be solved in time $O(n^k)$ for some constant k. Where n is the size of the input to the problem.

(ii) NP

NP stands for Non deterministic Polynomial Time

NP, where deterministic is just fancy way of talking about guessing a soln. A problem is in NP if you can quickly (in polynomial time) test whether a soln is correct.

(iii) Partition Problem:- Given n positive integers, determine if it is possible to partition them into two disjoint sub sets which have equal sum.

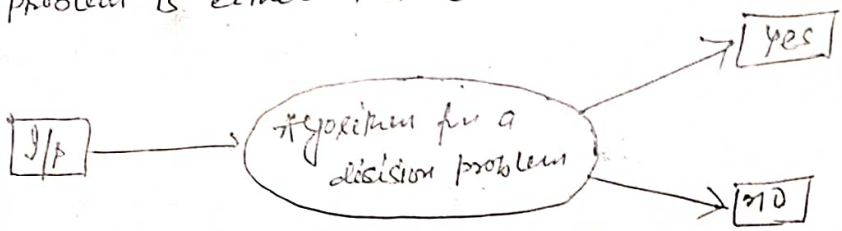
Graph Coloring - For a given graph find the smallest no. of colors that need to be assigned to the graph vertices so that no two adjacent vertices share the same color.

NP-Complete:- An NPC is an NP-problem. It is at least as difficult as an NP problem.

A problem that is NP-complete has the property that it can be solved in polynomial time, iff all other NP-complete problems can be solved in polynomial time.

Decision Problems:- A decision problem is one that asks only for a yes or no ans.

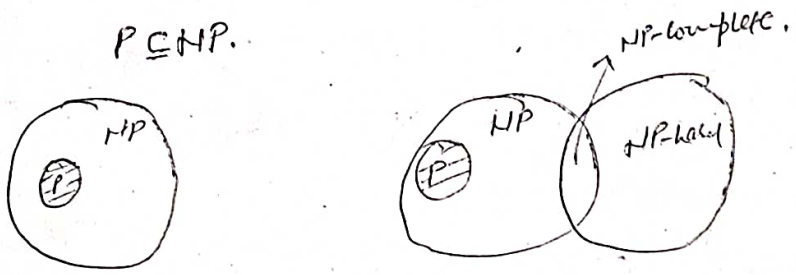
i.e the o/p of an algorithm for a decision problem is either 1 or 0.



NP-Hard- A problem L is NP-hard iff satisfiability reduces to L or any other NP-hard problem reduces to L. Means that L and satisfiability problems are polynomially equivalent.

Relationship b/w P & NP

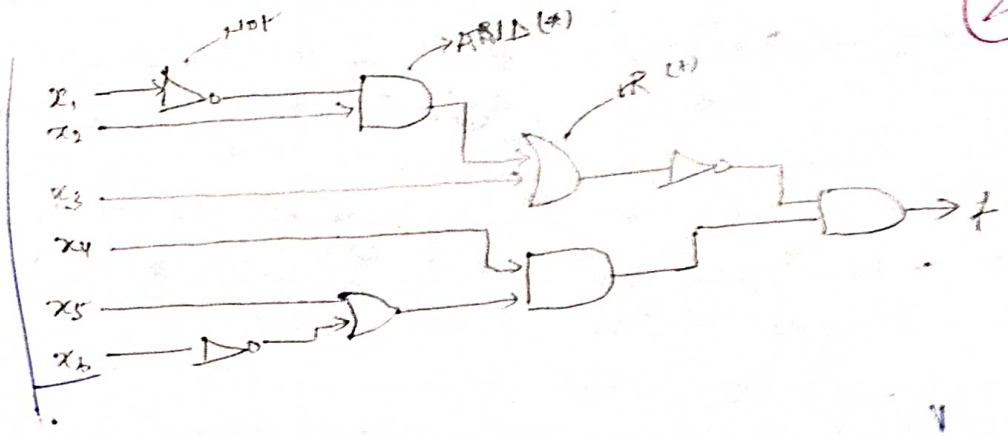
$P \subseteq NP$.

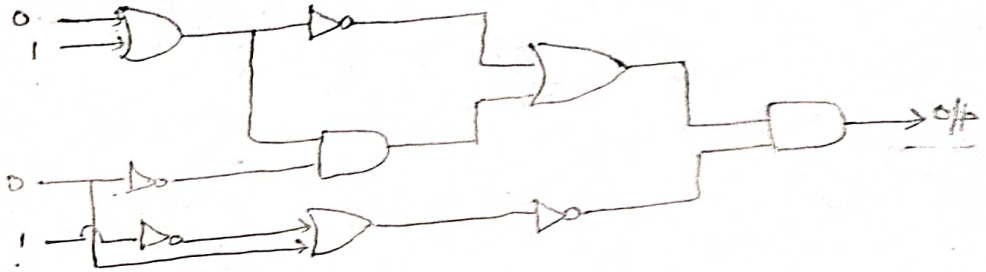


Circuit Satisfiability -

A combination ckt is called satisfiable if for set of inputs applied, o/p of the ckt is always 1.

CKT-SAT- Given a boolean combinational ckt composed of AND, OR & NOT gates, is it satisfiable?





CLIQUE PROBLEM

A clique in an undirected graph $G=(V,E)$ is a subset ~~of~~ $V' \subseteq V$ of vertices, such that for every two vertices in V', there exists an edge containing the two. Alternatively, a clique is a graph in which every vertex is connected to every other vertex in the graph.

In other words a clique is complete subgraph of G. The size of a clique is the no. of vertices it contains.

The problem of finding the size of a clique for a given graph is an ~~optiz~~ optimiz^n problem.

To prove clique problem is NP-complete, steps are

1. We use the set V', a subset of vertices in the clique. Checking whether V' is a clique or not can be done in polynomial time.

2. We prove 3-CNF SAT $\leq_p$ Clique, for this will construct a graph for the 3-CNF formula f:
   - Each literal becomes a vertex
   - No edge can be drawn from a literal to its negation.
   - Put an edge b/w 2 vertices V_i^r & V_j^s if they belong to diff. clauses.

Cook's Theorem -

Cook's theorem states that any NP-problem can be converted ~~to~~ to SAT in polynomial time.

In order to prove this we require a uniform way of representing NP problems.

Thus under Cook's theorem we will try to prove that CKT SAT is NP-complete, for this, CKT SAT ~~it~~ should be ~~in NP~~

3-CNF SAT Problem:

A boolean formula is in 3 CNF if it is conjunction (AND) of clause which are disjunction (OR) of exactly 3 literals.

"3 CNF ~~f~~ SAT problem states that "Given a 3 CNF formula, is it satisfiable?"

For reducing SAT to 3CNFSAT, i.e to show SAT $\leq_p$ 3-CNF SAT we use the following steps -

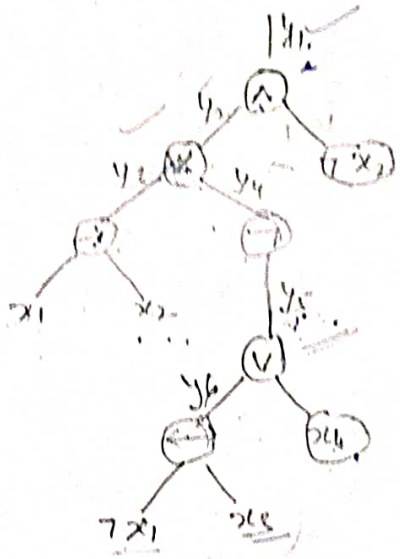
Step - 1. Construct a binary parse tree for the I/P formula f with literals as leaves and connectives as internal nodes. Apply associativity such that each node has at most two children. We associate a variable y_i to each internal node to hold its

Step-2 Rewrite the original formula as the AND of and and a conjunction of clauses describing the operation of each node. Call it f'.

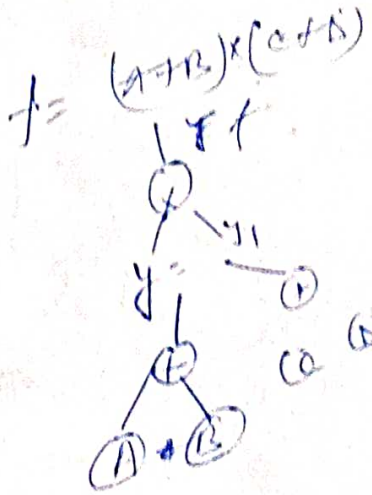
Step-3 – Convert each clause into CNF.

Step-4 – Transform the formula so that each clause contains exactly 3 literals.

Eg $f = ((x_1 \to x_2) \vee \neg((x_1 \leftrightarrow x_2) \vee x_4)) \wedge \neg x_1$



→ Binary Parse Tree.



or $f' = y_1 \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_1))$

$\wedge (y_2 \leftrightarrow (y_3 \vee y_4))$

$\wedge (y_3 \leftrightarrow (x_1 \to x_2))$

$\wedge (y_4 \leftrightarrow \neg y_5)$

$\wedge (y_5 \leftrightarrow y_6 \vee x_4)$

$\wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_2))$

$y_1 \wedge$

VERTEX COVER PROBLEM

A vertex cover of an undirected graph $G(V, E)$ is a subset V' of V, such that if $(u,v) \in E$ then $u \in V'$ or $v \in V'$ or both.

It is a set of vertices which cover all edges in E. The size of vertex cover is the no. of vertices in it.

In vertex cover problem we find a vertex cover of min^m size.

To show that vertex cover is NP—

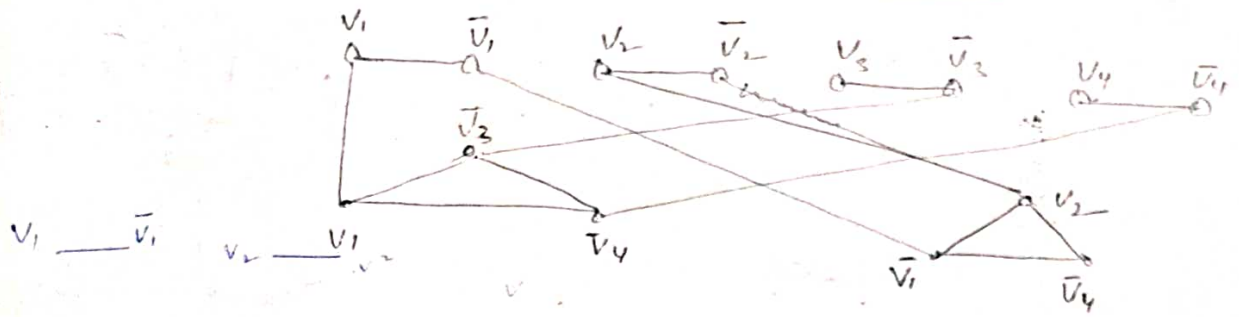
① first we take the subset V' as the certificate, & decide whether it covers all the edges by checking all edges in polynomial time time.

② Secondly we show that vertex cover in NP-complete, we reduce 3-CNF SAT to it. That is we prove—

3CNF SAT $\leq_p$ Vertex cover. For this formula we construct a graph using the following steps—

* Construct a graph with 2 vertices/variable.
* Construct 3 vertices for each clause (one for each literal), indicating the literals in clause.
* Connect all vertices which are from same variable
* Connect all vertices which are from same clause.

E.g $f = (x_1 \vee \overline{x_2} \vee \overline{x_4}) \wedge (x_2 \vee \overline{x_1} \vee \overline{x_4})$



APPROXIMATION ALGORITHM

"An algorithm that returns near optimal solutions is called an approximation algorithm"

Approximation algorithm is a procedure that always provides some kind of soln to your problem, though it may fail to find the optimal soln.

Vertex Cover Problem :-

A vertex cover of an undirected graph $G = (V, E)$, ~~find a vertex cover of~~ is a subset W of V ($W \subseteq V$) such that for every (a, b) in E, a is in W or b is in W.

Set-Cover $(S_1 \ldots S_m)$

Step 1- $C = \phi$

2 - $U = S$

3. Until U is empty, repeat step 4 to 6

4. Pick set S_i with max $|S_i \cap U|$

5. $U = U - S_i$

6. $C = C \cup \{S_i\}$

7. return C.

Eg

$S = \{a, b, c, d, e, f, g, h\}$

$S_1 = \{a, b, c\}$

$S_2 = \{b, d, f, g\}$

$S_3 = \{a, f, g\}$

$S_4 = \{c, d, e, h\}$

$S_5 = \{a, h\}$

$S_6 = \{b, c, f, h\}$

Soln

Initially,

$C = \phi$, $U = S = \{a, b, c, d, e, f, g, h\}$

Pick S_i with max $|S_i \cap U| = S_2$

$U = U - S_2 = \{a, c, e, h\}$

Pick S_i with max $|S_i \cap U| = S_1$

$C = C \cup \{S_1\} = \{S_1, S_2\}$

$U = U - S_1 = \{e, h\}$

Pick S_i with max $|S_i \cap U| = S_4$

~~$U = U - S_4$~~

$C = C \cup \{S_4\} = \{S_1, S_2, S_4\}$

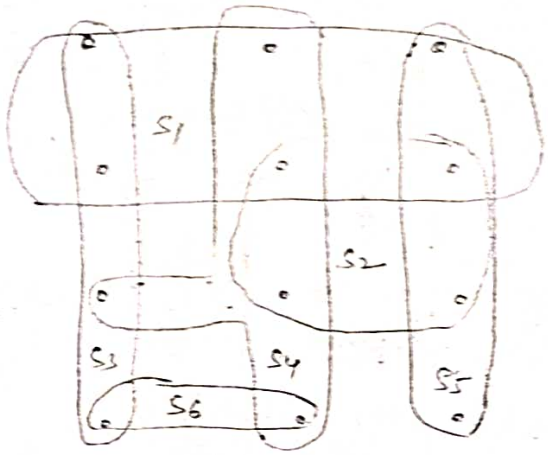
$U = U - S_4 = \phi$

So .. Set Cover $\{S_1, S_2, S_4\}$

3. U ← Union ~~of~~ of $S_1 \ldots S_m$
4. While U is not empty.
5. S_i ← Set in F with most elements in U
6. ~~C.add(S_i)~~ F.remove(S_i)
7. C.add(S_i)
8. Remove all elements in S_i from U
9. Return C.

E.g

X = 10 black points
$F = \{S_1, S_2, S_3, S_4, S_5, S_6\}$



Min^m size set cover is $C = \{S_3, S_4, S_5\}$

Alternate Algos for Vertex Cover & Set Cover

Vertex_Cover(G)
1. C = φ
2. Until there is no edge left in E, repeat 3 to 5
3. Pick an edge (u,v) arbitrarily
4. Remove all edges from E of the form (u,w) or (v,w)
5. C = C ∪ (u,v)
6. Return C.